

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- Constr |
- [Method](#)

- Detail:
- Field |
- Constr |
- [Method](#)

Interface Device

- All Known Implementing Classes:
[Headphones](#), [Laptop](#), [SmartPhone](#), [SmartWatch](#), [TV](#)

```
public interface Device
```

- Method Summary**

All Methods [Instance Methods](#) [Abstract Methods](#)

Modifier and Type	Method	Description
java.lang.String	Category()	
java.lang.String	Name()	
double	Price()	
int	Quantity()	
void	setName(java.lang.String name)	
void	setPrice(double price)	
void	setQuantity(int quantity)	

- Method Detail**

- Category**

```
java.lang.String Category()
```

- Name**

```
java.lang.String Name()
```

- setName**

```
void setName(java.lang.String name)
```

- Price**

```
double Price()
```

- setPrice**

```
void setPrice(double price)
```

- Quantity**

int Quantity()

▪ **setQuantity**

void setQuantity(int quantity)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

• [All Classes](#)

- Summary:
- Nested |
- Field |
- Constr |
- [Method](#)

- Detail:
- Field |
- Constr |
- [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class TV

- [java.lang.Object](#)
 - [TV](#)
- All Implemented Interfaces:
 - [Device](#)

```
public class TV
extends java.lang.Object
implements Device
```

- ◦ **Constructor Summary**

Constructors	
Constructor	Description
TV (java.lang.String name, double price, int quantity)	

- ◦ **Method Summary**

All Methods Instance Methods Concrete Methods		
Modifier and Type	Method	Description
java.lang.String	Category ()	O(1)
java.lang.String	Name ()	O(1)
double	Price ()	O(1)
int	Quantity ()	O(1)
void	setName (java.lang.String name)	O(1)
void	setPrice (double price)	O(1)
void	setQuantity (int quantity)	O(1)

- **Methods inherited from class java.lang.Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

- ◦ **Constructor Detail**

- **TV**

```
public TV(java.lang.String name,
double price,
int quantity)
```

- ◦ **Method Detail**

▪ **Category**

public java.lang.String Category()

O(1)

Specified by:

[Category](#) in interface [Device](#)

▪ **Name**

public java.lang.String Name()

O(1)

Specified by:

[Name](#) in interface [Device](#)

▪ **setName**

public void setName(java.lang.String name)

O(1)

Specified by:

[setName](#) in interface [Device](#)

▪ **Price**

public double Price()

O(1)

Specified by:

[Price](#) in interface [Device](#)

▪ **setPrice**

public void setPrice(double price)

O(1)

Specified by:

[setPrice](#) in interface [Device](#)

▪ **Quantity**

public int Quantity()

O(1)

Specified by:

[Quantity](#) in interface [Device](#)

▪ **setQuantity**

public void setQuantity(int quantity)

O(1)

Specified by:

[setQuantity](#) in interface [Device](#)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
- Nested |

- Field |
 - [Constr](#) |
 - [Method](#)
-
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class Laptop

- java.lang.Object
 - Laptop
- All Implemented Interfaces:
 - [Device](#)

```
public class Laptop
extends java.lang.Object
implements Device
```

- ◦ **Constructor Summary**

Constructors	
Constructor	Description
Laptop (java.lang.String name, double price, int quantity)	

- ◦ **Method Summary**

All Methods Instance Methods Concrete Methods		
Modifier and Type	Method	Description
java.lang.String	Category ()	O(1)
java.lang.String	Name ()	O(1)
double	Price ()	O(1)
int	Quantity ()	O(1)
void	setName (java.lang.String name)	O(1)
void	setPrice (double price)	O(1)
void	setQuantity (int quantity)	O(1)

▪ **Methods inherited from class java.lang.Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

- ◦ **Constructor Detail**

▪ **Laptop**

```
public Laptop(java.lang.String name,
              double price,
              int quantity)
```

- ◦ **Method Detail**

▪ **Category**

public java.lang.String Category()

O(1)

Specified by:

[Category](#) in interface [Device](#)

▪ **Name**

public java.lang.String Name()

O(1)

Specified by:

[Name](#) in interface [Device](#)

▪ **setName**

public void setName(java.lang.String name)

O(1)

Specified by:

[setName](#) in interface [Device](#)

▪ **Price**

public double Price()

O(1)

Specified by:

[Price](#) in interface [Device](#)

▪ **setPrice**

public void setPrice(double price)

O(1)

Specified by:

[setPrice](#) in interface [Device](#)

▪ **Quantity**

public int Quantity()

O(1)

Specified by:

[Quantity](#) in interface [Device](#)

▪ **setQuantity**

public void setQuantity(int quantity)

O(1)

Specified by:

[setQuantity](#) in interface [Device](#)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
- Nested |

- Field |
 - [Constr](#) |
 - [Method](#)
-
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class Headphones

- [java.lang.Object](#)
 - [Headphones](#)
- All Implemented Interfaces:
 - [Device](#)

```
public class Headphones
extends java.lang.Object
implements Device
```

- ◦ **Constructor Summary**

Constructors

Constructor	Description
Headphones (java.lang.String name, double price, int quantity)	

- ◦ **Method Summary**

All Methods [Instance Methods](#) [Concrete Methods](#)

Modifier and Type	Method	Description
java.lang.String	Category ()	O(1)
java.lang.String	Name ()	O(1)
double	Price ()	O(1)
int	Quantity ()	O(1)
void	setName (java.lang.String name)	O(1)
void	setPrice (double price)	O(1)
void	setQuantity (int quantity)	O(1)

▪ **Methods inherited from class java.lang.Object**

[clone](#), [equals](#), [finalize](#), [getClass](#), [hashCode](#), [notify](#), [notifyAll](#), [toString](#), [wait](#), [wait](#), [wait](#)

- ◦ **Constructor Detail**

▪ **Headphones**

```
public Headphones(java.lang.String name,
                  double price,
                  int quantity)
```

- ◦ **Method Detail**

▪ **Category**

public java.lang.String Category()

O(1)

Specified by:

[Category](#) in interface [Device](#)

▪ **Name**

public java.lang.String Name()

O(1)

Specified by:

[Name](#) in interface [Device](#)

▪ **setName**

public void setName(java.lang.String name)

O(1)

Specified by:

[setName](#) in interface [Device](#)

▪ **Price**

public double Price()

O(1)

Specified by:

[Price](#) in interface [Device](#)

▪ **setPrice**

public void setPrice(double price)

O(1)

Specified by:

[setPrice](#) in interface [Device](#)

▪ **Quantity**

public int Quantity()

O(1)

Specified by:

[Quantity](#) in interface [Device](#)

▪ **setQuantity**

public void setQuantity(int quantity)

O(1)

Specified by:

[setQuantity](#) in interface [Device](#)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
- Nested |

- Field |
 - [Constr](#) |
 - [Method](#)
-
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class SmartPhone

- [java.lang.Object](#)
 - [SmartPhone](#)
- All Implemented Interfaces:
 - [Device](#)

```
public class SmartPhone
extends java.lang.Object
implements Device
```

- ◦ **Constructor Summary**

Constructors

Constructor	Description
SmartPhone (java.lang.String name, double price, int quantity)	

- ◦ **Method Summary**

All Methods [Instance Methods](#) [Concrete Methods](#)

Modifier and Type	Method	Description
java.lang.String	Category ()	O(1)
java.lang.String	Name ()	O(1)
double	Price ()	O(1)
int	Quantity ()	O(1)
void	setName (java.lang.String name)	O(1)
void	setPrice (double price)	O(1)
void	setQuantity (int quantity)	O(1)

▪ Methods inherited from class java.lang.Object

[clone](#), [equals](#), [finalize](#), [getClass](#), [hashCode](#), [notify](#), [notifyAll](#), [toString](#), [wait](#), [wait](#), [wait](#)

- ◦ **Constructor Detail**

▪ SmartPhone

```
public SmartPhone(java.lang.String name,
                  double price,
                  int quantity)
```

- ◦ **Method Detail**

▪ **Category**

public java.lang.String Category()

O(1)

Specified by:

[Category](#) in interface [Device](#)

▪ **Name**

public java.lang.String Name()

O(1)

Specified by:

[Name](#) in interface [Device](#)

▪ **setName**

public void setName(java.lang.String name)

O(1)

Specified by:

[setName](#) in interface [Device](#)

▪ **Price**

public double Price()

O(1)

Specified by:

[Price](#) in interface [Device](#)

▪ **setPrice**

public void setPrice(double price)

O(1)

Specified by:

[setPrice](#) in interface [Device](#)

▪ **Quantity**

public int Quantity()

O(1)

Specified by:

[Quantity](#) in interface [Device](#)

▪ **setQuantity**

public void setQuantity(int quantity)

O(1)

Specified by:

[setQuantity](#) in interface [Device](#)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
- Nested |

- Field |
 - [Constr](#) |
 - [Method](#)
-
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class SmartWatch

- [java.lang.Object](#)
 - [SmartWatch](#)
- All Implemented Interfaces:
 - [Device](#)

```
public class SmartWatch
extends java.lang.Object
implements Device
```

- ◦ **Constructor Summary**

Constructors

Constructor	Description
SmartWatch (java.lang.String name, double price, int quantity)	

- ◦ **Method Summary**

All Methods [Instance Methods](#) [Concrete Methods](#)

Modifier and Type	Method	Description
java.lang.String	Category ()	O(1)
java.lang.String	Name ()	O(1)
double	Price ()	O(1)
int	Quantity ()	O(1)
void	setName (java.lang.String name)	O(1)
void	setPrice (double price)	O(1)
void	setQuantity (int quantity)	O(1)

▪ Methods inherited from class java.lang.Object

[clone](#), [equals](#), [finalize](#), [getClass](#), [hashCode](#), [notify](#), [notifyAll](#), [toString](#), [wait](#), [wait](#), [wait](#)

- ◦ **Constructor Detail**

▪ SmartWatch

```
public SmartWatch(java.lang.String name,
                  double price,
                  int quantity)
```

- ◦ **Method Detail**

▪ **Category**

public java.lang.String Category()

O(1)

Specified by:

[Category](#) in interface [Device](#)

▪ **Name**

public java.lang.String Name()

O(1)

Specified by:

[Name](#) in interface [Device](#)

▪ **setName**

public void setName(java.lang.String name)

O(1)

Specified by:

[setName](#) in interface [Device](#)

▪ **Price**

public double Price()

O(1)

Specified by:

[Price](#) in interface [Device](#)

▪ **setPrice**

public void setPrice(double price)

O(1)

Specified by:

[setPrice](#) in interface [Device](#)

▪ **Quantity**

public int Quantity()

O(1)

Specified by:

[Quantity](#) in interface [Device](#)

▪ **setQuantity**

public void setQuantity(int quantity)

O(1)

Specified by:

[setQuantity](#) in interface [Device](#)

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
- Nested |

- Field |
 - [Constr](#) |
 - [Method](#)
-
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

• [All Classes](#)

SEARCH:

reset

- Summary:
 - [Nested](#) |
 - [Field](#) |
 - [Constr](#) |
 - [Method](#)
- Detail:
 - [Field](#) |
 - [Constr](#) |
 - [Method](#)

Class Inventory

- java.lang.Object
 - Inventory

public class Inventory
extends java.lang.Object

- **Constructor Summary**

Constructors

Constructor Description

[Inventory\(\)](#)

- **Method Summary**

All Methods [Instance Methods](#) [Concrete Methods](#)

Modifier and Type	Method	Description
void	addDevice()	In this method, we iterate through the ArrayLists of the linked list with an enhanced for loop then adding the desired Device according to its category. In this CalculateTotal method, there is an nested enhanced for loop just like in previous methods.In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that
double	CalculateTotal()	ArrayList(category) then for each Device object we calculate its value by multiplying its price with its quantity and keep this value in total_value variable.After calculating every devices' value in total_value I returned total_value variable.
void	Export()	In this Export method, there is an nested enhanced for loop just like in previous methods.In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and display all the device objects with their categories,names,prices and quantities.
Device	FindCheapest()	In this FindCheapest method, there is an nested enhanced for loop just like in previous methods.In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category).Before the loop I created a Device object to keep the CheapestDevice object.The first object is a dummy temporary object and it is changed with the first Device object in the loop in ' CheapestDevice.Price()=-1' since the price of the temp object is -1.
void	ListAll()	In this ListAll method, there is an nested enhanced for loop just like in previous methods.In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and I display all the devices.

void [removeDevice\(\)](#)

In this remove method, there is an nested enhanced for loop. In the outer loop I iterate through the ArrayLists(categories) and in the inner loop I iterate through the Device objects in that ArrayList(category) and if the device is found I remove that device.

void [Restock\(\)](#)

In this Restock method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category). Before that the device name is wanted from the user then if the stock will be added or removed is asked from the user then the values. According to that values after the device is found in the for loop its stock value is changed.

void [SortDevices\(\)](#)

In this SortDevices method, there is two nested for loops. In first for loops, in the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category). In the first nested for loop I stored all the Devices by adding them to a ArrayList to sort them using this ArrayList. In the second nested for loop I used two loops first for iterating through every element and second to compare every element after that element for sorting. In that second loop if the next element is smaller than the current element(price of the Device object) smaller object is kept in another object before the removal then it is removed and added to the place where the current element is placed. This sorting mechanism occurs for every element in the ArrayList.

void [updateDevice\(\)](#)

In this update method, there is an nested enhanced for loop just like in remove method. In the outer loop I iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and if the device to be updated is found I update that device with the desired values.

▪ Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

• ◦ Constructor Detail

▪ Inventory

```
public Inventory()
```

◦ Method Detail

▪ addDevice

```
public void addDevice()
```

In this method, we iterate through the ArrayLists of the linked list with an enhanced for loop then adding the desired Device according to its category. If the category of the device is not found a new ArrayList is added for that category and device is added to that category(ArrayList). If and only if the category is one of the 5 categories(TV, Laptop, SmartPhone, Smart Watch, Headphones). Time complexity of this method is $O(n)$ where 'n' is the length of the linked list which is the number of ArrayLists(categories) in the linked list. It is because there is a for loop iterates through our linked list which has time complexity of $O(n)$ there is also a bunch of if-else statements However since each of them is $O(1)$ it does not effect since there is $O(n)$. Time complexity : $O(n)$.

▪ removeDevice

```
public void removeDevice()
```

In this remove method, there is an nested enhanced for loop. In the outer loop I iterate through the ArrayLists(categories) and in the inner loop I iterate through the Device objects in that ArrayList(category) and if the device is found I remove that device. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. However, the remove method of Java has a time complexity of $O(n)$ so the worst time complexity of this method is assuming length as 'd' d(from nested enhanced for loop) times d(remove method). Time complexity : $O((n*m)^2)$ or $O(d^2)$.

▪ updateDevice

```
public void updateDevice()
```

In this update method, there is an nested enhanced for loop just like in remove method. In the outer loop I iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and if the device to be updated is found I update that device with the desired values. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and

Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. The constant setting operations has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O(n*m)$ or $O(d)$.

■ ListAll

```
public void ListAll()
```

In this ListAll method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and I display all the devices. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. The constant getting operations(devices.Category() etc.) has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O(n*m)$ or $O(d)$.

■ FindCheapest

```
public Device FindCheapest()
```

In this FindCheapest method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category). Before the loop I created a Device object to keep the CheapestDevice object. The first object is a dummy temporary object and it is changed with the first Device object in the loop in '|| CheapestDevice.Price() == -1' since the price of the temp object is -1. After that if any of the other devices is cheaper the CheapestDevice object is replaced with that object and finally that object is returned. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. The constant if statement has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O(n*m)$ or $O(d)$.

Returns:

CheapestDevice(Device object)

■ SortDevices

```
public void SortDevices()
```

In this SortDevices method, there is two nested for loops. In first for loops, in the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category). In the first nested for loop I stored all the Devices by adding them to a ArrayList to sort them using this ArrayList. In the second nested for loop I used two loops first for iterating through every element and second to compare every element after that element for sorting. In that second loop if the next element is smaller than the current element(price of the Device object) smaller object is kept in another object before the removal then it is removed and added to the place where the current element is placed. This sorting mechanism occurs for every element in the ArrayList. The time complexity of the first nested for loop which is an enhanced for loop used in previous methods is ; If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. The time complexity of the second for loop which is the loop used for sorting the Device objects is ; The length of the SortedList which includes all the devices is actually $n*m$. It comes from the first nested for loop since it iterates through every device in the linked list (n categories * m average device in a category = $n*m$ devices). Therefore in the second nested for loop we iterate through the devices and the next element of the devices in the inner loop it results in ; $(n*m)^2$ time complexity in overall since it both loops iterates through the ArrayList. The inner loop iterates through the ArrayList less since it starts from current index + 1. However it still results in time complexity similar to $(n*m)$ in that loop. Therefore the time complexity is $O((n*m)^2)$ In addition, removing method in the second nested loop also has a time complexity of $O(n)$ which is $O(n*m)$ therefore the overall worst time complexity is $O((n*m)^2)$ [from 2nd nested for loop] times $O(n*m)$ [from remove method] which is $O((n*m)^3)$ if we assume $n*m$ as total device number as 'd' shortly the time complexity is $O(d^3)$. The constant adding, getters and setters has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O((n*m)^3)$ or $O(d^3)$.

■ CalculateTotal

```
public double CalculateTotal()
```

In this CalculateTotal method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) then for each Device object we calculate its value by

multiplying its price with its quantity and keep this value in total_value variable. After calculating every devices' value in total_value I returned total_value variable. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. The constant operations(calculations and getters) has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. Time complexity : $O(n*m)$ or $O(d)$.

Returns:
total_value

■ Restock

```
public void Restock()
```

In this Restock method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category). Before that the device name is wanted from the user then if the stock will be added or removed is asked from the user then the values. According to that values after the device is found in the for loop its stock value is changed. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. The constant operations(if statements, calculations, setters and getters) has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O(n*m)$ or $O(d)$.

■ Export

```
public void Export()
```

In this Export method, there is an nested enhanced for loop just like in previous methods. In the outer loop we iterate through the ArrayLists(categories) and in the inner loop we iterate through the Device objects in that ArrayList(category) and display all the device objects with their categories, names, prices and quantities. If we assume the length of the linked list as 'n' and assume the average device numbers in categories as 'm' the time complexity is $O(n*m)$ since the for loops iterates through the linked list(n) and Devices in the ArrayLists(m) resulting in $n*m$ time complexity. Also if we assume the total number of devices as 'd' then the time complexity will be $O(d)$ which is the same as $O(n*m)$. The constant getting operations(devices.Category() etc.) has $O(1)$ time complexity therefore does not effect the time complexity much. In conclusion ; Time complexity : $O(n*m)$ or $O(d)$.

[Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
- [All Classes](#)
- Summary:
 - Nested |
 - Field |
 - [Constr](#) |
 - [Method](#)
- Detail:
 - Field |
 - [Constr](#) |
 - [Method](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)

- [All Classes](#)

SEARCH:

reset

- Summary:
- Nested |
- Field |
- [Constr](#) |
- [Method](#)

- Detail:
- Field |
- [Constr](#) |
- [Method](#)

Class TheMenu

- java.lang.Object
- ◦ TheMenu

- ```
public class TheMenu
 extends java.lang.Object
```

- ◦ **Constructor Summary**

Constructors

### Constructor Description

[TheMenu\(\)](#)

- ◦ **Method Summary**

All Methods [Static Methods](#) [Concrete Methods](#)

| Modifier and Type | Method                                                                    | Description                                                                                                                                                            |
|-------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| static void       | <a href="#">main</a> (java.lang.String[] args)                            |                                                                                                                                                                        |
| static void       | <a href="#">TheMenu</a><br>( <a href="#">Inventory</a> ElecShopInventory) | The menu of the electronic shop inventory using a while loop till '0' is entered giving user the corresponding action regarding the entered number from ther user(1-9) |

- **Methods inherited from class java.lang.Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

- ◦ **Constructor Detail**

- **TheMenu**

```
public TheMenu()
```

- ◦ **Method Detail**

- **main**

```
public static void main(java.lang.String[] args)
```

- **TheMenu**

```
public static void TheMenu(Inventory ElecShopInventory)
```

The menu of the electronic shop inventory using a while loop till '0' is entered giving user the corresponding action regarding the entered number from ther user(1-9)

Parameters:

ElecShopInventory -

### [Skip navigation links](#)

- [Package](#)
- [Class](#)
- [Use](#)
- [Tree](#)
- [Deprecated](#)
- [Index](#)
- [Help](#)
  
- [All Classes](#)
  
- Summary:
- [Nested](#) |
- [Field](#) |
- [Constr](#) |
- [Method](#)
  
- Detail:
- [Field](#) |
- [Constr](#) |
- [Method](#)